Open in Colab

`for` Statements

In Python tuples, lists, sets, as well as dictionaries and strings are iterable objects.

`for` statements are used to iterate over some iterable objects.

In [1]:
```
fib = 0,1,1,2,3,5,8,13,21,
```

In this example we have a tuple `fib`. You can get each item in the list with `for` statement.

In [2]:
```
for i in fib:
  print(i)
```

```
0
1
1
2
3
5
8
13
21
```

In each iteration the variable `i` becomes the next item. Of course you can use any valid variable name. But if you already assigned a value to that variable it will be overriden.

Like `if` and `while` statements, `for` statements can also have an optional `else` statement. It will be evaluated at the end of the loop.

In [3]:
```
for i in fib:
  print(i)
else:
  print("finished")
```

```
0
1
1
2
3
5
8
13
21
finished
```

Breaking The Loops

Sometimes you may want to exit from the loop even the condition is still `True`. To do use `break`. If you break the loop then `else` statement is not evaluated.

In [4]:
```
for i in fib:
  if i > 10:
    break
  print(i)
```

```
else:
  print("finished")
```

```
0
1
1
2
3
5
8
```

`continue` statement is used to go to the start of the loop without evaluating the remaining of the loop.

In [5]:
```
for i in fib:
  if i < 5:
    continue
  if i > 10:
    break
  print(i)
else:
  print("finished")
```

```
5
8
```

Nested Loops

When using `break` inside a nested loop, you can break only the inner one.

In [6]:
```
for i in fib:
  for j in fib:
    break
  print("Only the inner loop is break.")
  break
```

```
Only the inner loop is break.
```

Exercise: Iterating Over Strings

In [7]:
```
for a in 'hello world!':
  print(a,ord(a))
```

```
h 104
e 101
l 108
l 108
o 111
  32
w 119
o 111
r 114
l 108
d 100
! 33
```

In [8]:
```
letters_used = set()
for letter in 'hello world':
  letters_used.add(letter)
print(letters_used)
```

```
{'e', 'w', 'r', ' ', 'h', 'd', 'l', 'o'}
```

In [9]:
```
letters_used = dict()
```

```
for letter in 'hello world':
  if letter in letters_used:
    letters_used[letter] += 1
    # letters_used[letter] = letters_used[letter] + 1
  else:
    letters_used[letter] = 1
print(letters_used)
```

```
{'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}
```

Exercise: Iterating Over Dictioanies

In [10]:
```
for letter in letters_used:
  print(letter, letters_used[letter])
```

```
h 1
e 1
l 3
o 2
  1
w 1
r 1
d 1
```

Warning: If you iterate over a dictionary you will get only the keys of the dictionary. To get the `key,value` pairs use `.items()`:

In [11]:
```
letters_used.items()
```

Out[11]:
```
dict_items([('h', 1), ('e', 1), ('l', 3), ('o', 2), (' ', 1), ('w', 1), ('r', 1), ('d', 1)])
```

In [12]:
```
for letter,count in letters_used.items():
  print(letter, count)
```

```
h 1
e 1
l 3
o 2
  1
w 1
r 1
d 1
```

or you can use `*` to unpack `key,value` pairs:

In [13]:
```
for letter_count in letters_used.items():
  print(*letter_count)
```

```
h 1
e 1
l 3
o 2
  1
w 1
r 1
d 1
```

`range()` Objects

In many `for` loops we need a counter starting with a given number. `range` function is used to produce counting numbers up to but not including given number.

In [14]:
```
for i in range(5):
  if i == 1:
    print(f"This is the {i}st step")
  elif i == 2:
    print(f"This is the {i}nd step")
  elif i == 3:
    print(f"This is the {i}rd step")
  else:
    print(f"This is the {i}th step")
```

```
This is the 0th step
This is the 1st step
This is the 2nd step
This is the 3rd step
This is the 4th step
```

In [15]:
```
for i in range(2,6):
  if i == 1:
    print(f"This is the {i}st step")
  elif i == 2:
    print(f"This is the {i}nd step")
  elif i == 3:
    print(f"This is the {i}rd step")
  else:
    print(f"This is the {i}th step")
```

```
This is the 2nd step
This is the 3rd step
This is the 4th step
This is the 5th step
```

In [16]:
```
for i in range(2,6,2):
  if i == 1:
    print(f"This is the {i}st step")
  elif i == 2:
    print(f"This is the {i}nd step")
  elif i == 3:
    print(f"This is the {i}rd step")
  else:
    print(f"This is the {i}th step")
```

```
This is the 2nd step
This is the 4th step
```

In [17]:
```
for i in range(6,2,-1):
  if i == 1:
    print(f"This is the {i}st step")
  elif i == 2:
    print(f"This is the {i}nd step")
  elif i == 3:
    print(f"This is the {i}rd step")
  else:
    print(f"This is the {i}th step")
```

```
This is the 6th step
This is the 5th step
This is the 4th step
This is the 3rd step
```

In [18]:
```
for _ in range(4):
  print("something")
```

```
something
something
something
something
```

Comprehensions (`for` and `for` - `if` Expressions)

`set` Comprehensions

You can create a set easily by using `for` expression inside a set.

In [19]:
```
{i**2 for i in range(5)}
```

Out[19]:
```
{0, 1, 4, 9, 16}
```

This is mathematically equivalent to the set:

$$\{i^2 \mid i = 0, 1, \dots, 4\}$$

If you want to filter only some of them you can use `for` - `if` expression.

In [20]:
```
{i**2 for i in range(5) if i & 1}
```

Out[20]:
```
{1, 9}
```

`list` Comprehensions

Just like set comprehension you can use `for` expression inside a list to create a list comprehension.

In [21]:
```
[i**2 for i in range(5)]
```

Out[21]:
```
[0, 1, 4, 9, 16]
```

`dict` Comprehensions

Dictionary comprehensions are also created with a similar usage.

In [22]:
```
{i:i**2 for i in range(5)}
```

Out[22]:
```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Exercises

In [23]:
```
for i in range(150):
  if i > 50:
    i = i - 50
  elif i % 2:
    i = i * 2
  else:
    i = i + 50

print(i)
```

```
99
```

In [24]:
```
PI = 3.14
a = 0
for i in range(100):
  a += PI
  if a > 10:
```

```
    i = 1000
print(i)
```

```
1000
```

In [25]:
```
for i in range(5):
  if i % 2:
    i *= 2
  if i > 10:
    i -= 10
print(i)
```

```
4
```

In [26]:
```
mylist = []

mymessage = ''

for i in mylist:
  mymessage = "hello"

print(mymessage)
```